

Week 3 Write Up for
Master of Science
Information and Communications Technology

Melissa Knapp
University of Denver University College

April 19, 2026

Faculty: Nathan Braun, MA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Table of Contents

Table of Contents i

Overview 1

Challenges Faced..... 1

JUnit Testing 2

Future Considerations..... 5

Overview

The overall problem being solved is how to track the visitors to various parking lots and charge them for time parked each month. The system, among other functions, registers customers, their cars, and tracks where and when they park. This week we created a layer in the system that would handle registering cars and customers.

I first started by created a shell for the new interface: "ParkingChargeStrategy". From there I tried to determine what strategies to implement. I decided that, along with the base strategy, to discount weekends in one strategy, and discount "special days" in another strategy.

Challenges Faced

One of the first challenges faced this week was how to track days for the weekend discount and special days for the special days strategy. Initially I was passing in an array of days and a Boolean value to indicate special day. But the more I thought about it, longer stays with only 1 special day would be impossible to calculate from this data. I ended up deciding to pass in a HashMap, where the key was the day of the week and the value was a Boolean indicating if the day was a special day.

The second challenge I faced this week was, after implementing and testing the strategy classes, refactoring the parking lot and parking charges to work with the new strategies. I had some trouble figuring out how the parking system would calculate the hours parked and generate the HashMap for my strategy classes. I finally hit upon the assumption that the LocalDateTime attribute in a parking charge was the time when the customer entered the parking lot and the Instant attribute was the time when the customer left the parking lot. From this I researched how to get the two variables into the same type so that I could compare them.

And from there I found the Duration variable which would help me separate out the days between the two times and total hours between the two times.

After I had resolved this issue, I figured that I could pull the day of the week from the LocalDateTime variable and then loop through the LocalDateTime, adding a single day until I reached the same day as the Instant. In that loop I grabbed the day of the week and formatted it to match how I had the string formatted in the weekend discount. And since I had set the base strategy as the one being used, I assumed all days would be false for special days. If I end up swapping to the special day discount, I would need to hard-code some special days into the program and then use a comparison to set the HashMap entry's value to true if the day in my loop matched a special day.

JUnit Testing

For all 3 JUnit's I used the same set of variables and calculated the charge by hand before running the test. I used a HashMap with 1 weekend day and 1 or 2 special days, depending on the test. And I set up 1 SUV and 1 Compact car to test the Compact discount. I ran into some trouble with the weekend and special day tests. I initially calculated the following for a total charge:

$$totalCharge = (baseRate * 24 * daysParked) + (baseRate * (hoursParked \% 24))$$

The problem with this formula is that when parked for more than a day it ended up charging more than I anticipated. For example if I pass in a HashMap of 3 days and a total hours parked of 56 hours. I expected the total charge without any discounts would be 56 times the base rate. And it ended up being 80 times the base rate. The easiest way to fix this would have been to just multiply the hours parked by the base rate, but this only works for the base

strategy, because it is calculated irrespective of day and day type. So what I ended up doing is adding an if statement into the 2 strategies that relied on days for discounts. And if daysParked was greater than 1 and the remainder of hourParked divided by 24 wasn't 0, then I subtracted a day from the non-discounted value. I also added a check to make sure that the non-discounted days was greater than 0, so that I didn't end up with negative days and in those cases I subtracted 1 day from the discounted days.

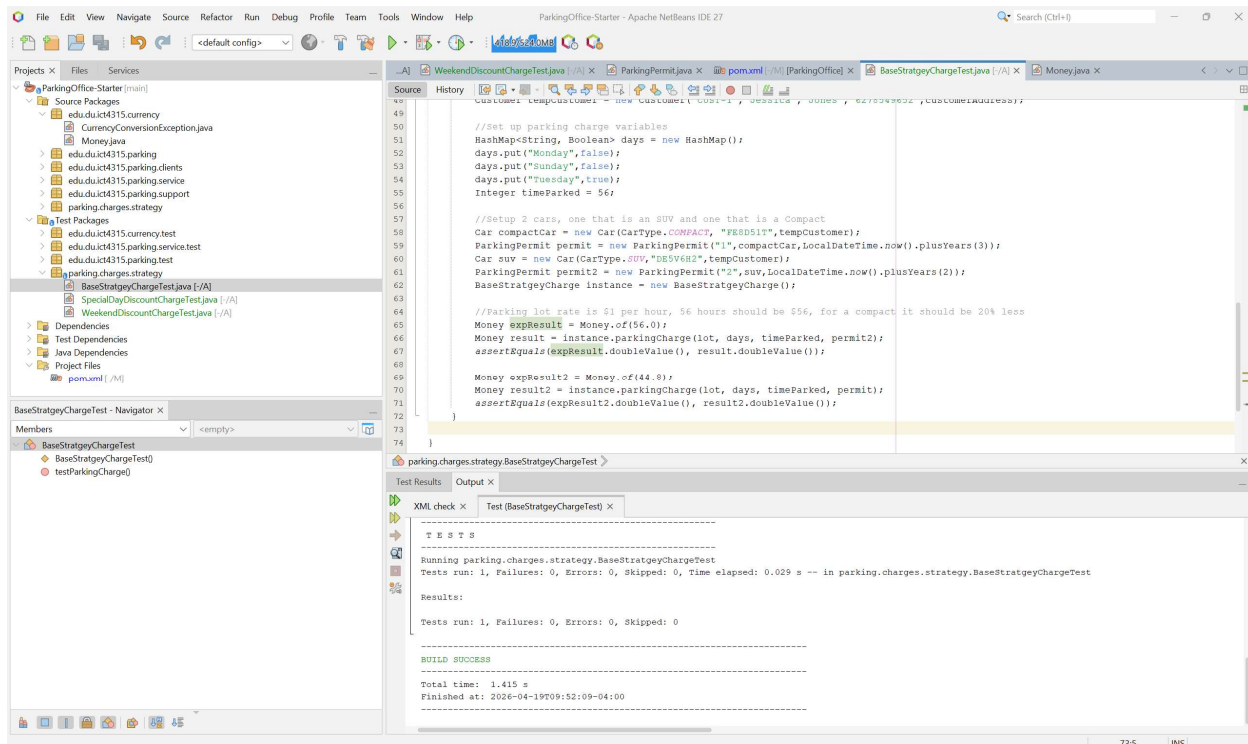


Figure 1. JUnit test results for BaseStrategyChargeTest

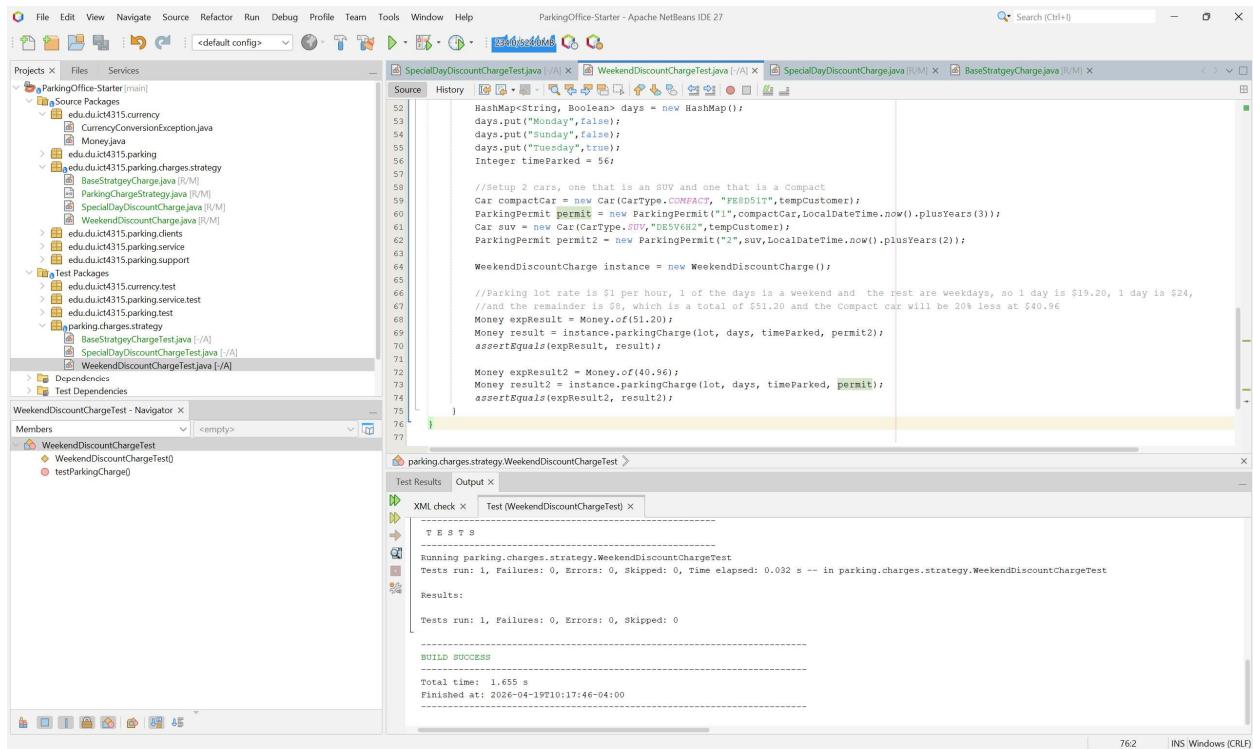


Figure 2. JUnit test results for WeekendDiscountChargeTest

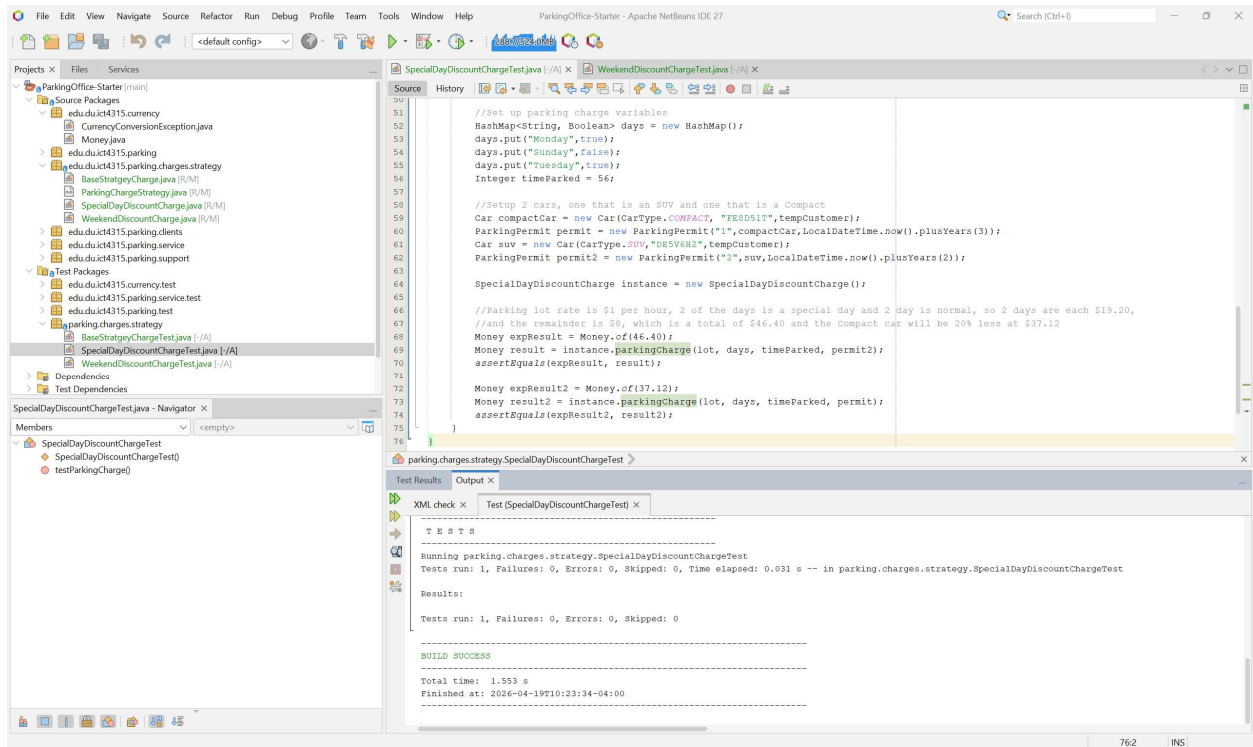


Figure 3. JUnit test results for SpecialDayDiscountChargeTest

Future Considerations

One of my ongoing objectives is to become proficient with the Logger library. I didn't spend much time in the last week learning about it and I didn't end up needing to throw exceptions or log any errors. So, I have held off on updating the registration classes to use Logger instead of Exceptions. But that is still on my to-do list as I want the codebase to have a consistent direction for error handling.